

Raritone Project Backend

1. Project Overview

Raritone Backend is a Node.js + Express.js REST API backend for a fashion/e-commerce platform that appears to focus on:

- User Authentication
 - Google Authentication
 - Avatar Customization
 - Wardrobe Management
 - Virtual Try-On Functionality
 - Product Catalog
 - Shopping Cart
 - Wishlist
 - Orders
 - Image Management
 - Body Measurements/Profile Management
- The backend

follows a layered architecture:

Client



Routes



Controllers



Models



MongoDB Database

Additional layers:

Middleware

Validation

Utilities

Clouinary Storage JWT

Authentication

2. Technology Stack

Backend Framework

- Node.js
- Express.js

Database

- MongoDB
- Mongoose ODM

Authentication

- JWT (JSON Web Token)
- Google OAuth

File Storage

- Clouinary

Validation

- Express Validator / Custom Validators

Security

- Authentication Middleware
- Role-Based Authorization
- Input Validation

3.Application Startup Flow

When the application starts:

Step 1

Express server is initialized.

Step 2

Middleware components are loaded:

- CORS
- Compression
- JSON Parser
- Rate Limiter

Step 3

MongoDB connection is established.

Step 4

All route modules are registered.

Step 5

Application begins listening for API requests.

4. Security Implementation

Security is a major component of the system.

JWT Authentication

After login, a JWT token is generated and sent to the client.

The client includes this token in future requests.

The backend validates the token before allowing access to protected resources.

Password Encryption

User passwords are never stored directly.

Before saving:

1. Password is received.
2. Salt is generated.
3. Password is hashed using bcrypt.
4. Hashed value is stored.

This protects user credentials even if the database is compromised.

Role-Based Access Control (RBAC)

Supported Roles:

USER

ADMIN

SUPER_ADMIN

Permissions

USER

- Browse products
- Manage cart
- Place orders

ADMIN

- Product management
- Inventory updates

SUPER_ADMIN

- System-level management
- Administrative operations

Helmet Protection

Helmet secures HTTP headers against:

- XSS attacks
- Clickjacking

- MIME-sniffing
- Protocol attacks

Rate Limiting

The system limits requests to:

100 requests per 15 minutes This

helps prevent:

- Brute force attacks
- API abuse
- Server overload

Input Validation

Joi validators are implemented to validate:

- Emails
- Passwords
- Product data
- Avatar requests
- Try-On requests

Secure Environment Variables

Sensitive information stored in:

JWT_SECRET
MONGO_URI
CLOUDINARY_API_KEY
CLOUDINARY_SECRET
GOOGLE_CLIENT_ID

Advantages:

- Source code protection
- Credential isolation

- Production security

5. Authentication Module

The Authentication Module manages user identity and access control.

• User Registration

When a new user signs up:

Process

1. User submits name, email, and password.
2. Backend validates input.
3. Existing email is checked.
4. Password is encrypted.
5. User record is created.
6. Success response is returned.

Endpoint

POST /api/auth/signup

• User Login Process

1. User enters email and password.
2. Backend verifies credentials.
3. Password hash is compared.
4. JWT token is generated.
5. Token is returned.

Endpoint

POST /api/auth/login

Forgot Password

Allows users to recover their accounts.

Process

1. User submits email.

2. Reset token is generated.
3. Email is sent.
4. User resets password using token.

Endpoints

POST/api/auth/forgot-password

POST /api/auth/reset-password/:token

Functionalities

- Email verification
- Password comparison
- JWT generation
- Refresh token support
- Session creation

• Google Authentication

The system supports Google OAuth login.

Flow

1. User signs in using Google.
2. Google provides authentication token.
3. Backend verifies token.
4. User account is created or fetched.
5. JWT token is issued.

6. Product Management Module

The Product Module manages clothing products available on the platform.

Purpose

This module allows administrators to create and manage products while users can browse and search them.

• **Add Product**

Administrators can upload new products.

Features

- Product validation
- Multiple image uploads
- Cloudinary storage
- Image compression using Sharp
- Optimized image URLs

Product Attributes

- Title
- Description
- Price
- Category
- Images
- Stock
- Seller Details

Process

1. Product details submitted.
2. Images compressed.
3. Images uploaded to Cloudinary.
4. Product stored in MongoDB.

Endpoint

POST /api/products

• **View Products**

Returns all available products.

Endpoint

GET /api/products

Supports:

- Filtering
- Searching
- Category-based retrieval
- Pagination

View Product Details

Returns information for a specific product.

Endpoint

GET /api/products/:id

• Update Product

Allows product modifications.

Endpoint

PUT /api/products/:id

Delete Product

Removes product and associated images.

Endpoint

DELETE /api/products/:id

7. Wishlist Module

The Wishlist Module enables users to save products for future purchases.

Purpose

Users often browse products without purchasing immediately. Wishlist functionality allows products to be stored for later consideration.

• Add Product to Wishlist

Process

1. Product ID received.

2. Wishlist searched.
3. Duplicate check performed.
4. Product added.

Endpoint

POST /api/wishlist

• Remove Product

Endpoint

DELETE /api/wishlist/:productId

Retrieve Wishlist Endpoint

GET /api/wishlist

8. Wardrobe Module

The Wardrobe Module acts as a virtual closet.

Users can store details about clothing they already own.

Features

Users can save:

- Clothing name
- Category
- Color
- Brand
- Size
- Condition
- Image

• Add Clothing Item

Process

1. User uploads clothing image.
2. Image stored in Cloudinary.

3. Metadata saved in MongoDB.

Endpoint

POST /api/wardrobe

. Get Wardrobe Items

Returns all wardrobe items belonging to a user.

Endpoint

GET /api/wardrobe

. Update Wardrobe Item

Allows clothing details to be modified.

Endpoint

PUT /api/wardrobe/:id

. Delete Wardrobe Item

Deletes wardrobe item and image.

Endpoint

DELETE /api/wardrobe/:id

9. Profile Module

The Profile Module stores personalized user information.

Features

Personal Information

Stores:

- Bio
- Profile picture
- Preferences **User Preferences** Stores:
- Theme
- Language
- Notification settings

Get Profile

GET /api/profile

Update Profile

PUT /api/profile

Update Preferences PUT

/api/profile/preferences

10. Cart Module

The Cart Module handles temporary shopping selections.

Purpose

Users can collect products before placing an order.

. Add Product to Cart

Process

1. Product selected.
2. Cart checked.
3. Product added or quantity updated.

Endpoint

POST /api/cart

. Get Cart

Returns complete cart contents.

Endpoint

GET /api/cart

. Remove Item

Deletes a product from cart.

Endpoint

DELETE /api/cart/:productid

Operations performed:

- Product validation

- Quantity validation
- Cart creation
- Existing item updates
- Price synchronization

11. Order Management Module

The Order Module manages completed purchases.

• Create Order

Process

1. User confirms purchase.
2. Product information submitted.
3. Total amount calculated.
4. Order record created.

Endpoint

POST /api/orders

• View Orders

Returns all orders belonging to a user.

Endpoint

GET /api/orders

• View Single Order

Returns detailed order information.

Endpoint

GET /api/orders/:id

12. Measurement Module

The Measurement Module stores body measurements.

This information can later be used for:

- Personalized recommendations

- Accurate sizing
- Virtual try-on systems

- **Measurements Stored**

- Chest
- Waist
- Shoulder
- Hip
- Height

These measurements are later utilized by AI recommendation and virtual try-on modules.

- **Save Measurements**

POST /api/measurements

- **Retrieve Measurements**

GET /api/measurements

13. Virtual Try-On Module

One of the most innovative features of the platform.

The Try-On Module allows users to compare original and generated clothing images.

Functionality Users

upload:

1. Original Image
2. Generated Try-On Image

Both images are stored in Cloudinary.

The system maintains a history of all try-on sessions.

- **Upload Try-On Images**

POST /api/tryOnRoutes

- **Get Try-On History**

GET /api/tryOnRoutes

- **Delete Try-On Record**

DELETE /api/tryOnRoutes/:id

14. Avatar Module

The Avatar Module manages user avatars.

Create Avatar

POST /api/avatar

Purpose:

- User avatar generation
- Body type mapping
- Skin tone mapping
- Gender configuration

Users can:

- Upload avatars
- Update avatars
- Delete avatars

Cloudinary storage ensures efficient image management.

15. Try-On APIs

Generate Virtual Try-On

POST /api/tryon

Process:

1. Upload body image
2. Upload selected clothing
3. AI processing

4. Generate output image
5. Save generated image URL
6. Return try-on result

16. Image Management Module

This module handles image uploads across the platform.

Supported Areas

- Product Images
- Wardrobe Images
- Profile Images
- Avatar Images
- Try-On Images

Image Optimization Process

Step 1

Image received.

Step 2

Sharp compresses image.

Step 3

Resolution adjusted.

Step 4

Cloudinary upload performed.

Step 5

Optimized URL returned.

This process improves:

- Performance
- Storage efficiency
- Loading speed

17. API Response Structure

Successful responses generally follow:

```
</> JSON

{
  "success": true,
  "message": "Operation completed",
  "data": {}
}
```

Error responses:

```
</> JSON

{
  "success": false,
  "message": "Error description"
}
```

18.Database schemas

The application uses MongoDB as the primary NoSQL database. MongoDB provides flexibility for handling rapidly evolving fashion, user profile, and AI-generated content data.

• User Collection

Purpose

Stores authentication and user management information.

Fields

Field	Type	Description
_id	ObjectId	Primary Key
name	String	User Name
email	String	Unique Email
password	String	Encrypted Password

role	String	USER/ADMIN
profileImage	String	Profile Picture
avatarImage	String	AI Avatar
bodyScanImage	String	Body Scan Image
createdAt	Date	Creation Timestamp

Relationships

User → Wishlist

User → Cart

User → Orders

User → Measurements

User → Profile

User → Avatar

User → TryOn

• Product Collection

Purpose

Stores product inventory.

Fields

Field	Type
title	String
description	String
price	Number
category	String
images	Array
stock	Number
sellerInfo	String

Relationships

Product → Wishlist

Product → Cart

Product → Orders

Product → Product Mapping

• Order Collection

Stores purchase history.

Fields

- userId
- productId
- quantity
- totalAmount
- shippingAddress
- orderStatus
- paymentStatus
- timestamps

• Cart Collection

Stores temporary shopping data.

Structure

```
{
  userId,
  products:[
    {
      productId,
      quantity
    }
  ]
}
```

- **Wishlist Collection**

Stores favorite products.

Benefits

- Quick retrieval
- Personalized recommendations
- User engagement tracking

- **Measurement Collection**

Stores body measurements.

Fields

- Chest
- Waist
- Hip
- Shoulder
- Height

Business Importance

These measurements enable future AI-driven size prediction and personalized recommendations.

- **Avatar Collection**

Stores generated avatar information.

Fields

- Avatar Image
- Gender
- Body Type
- Skin Tone
- Status

- **Try-On Collection**

Stores AI-generated virtual fitting results.

Fields

- Original Image
- Generated Image
- User ID
- Creation Date

19.Cloudinary workflow

Cloudinary serves as the primary cloud-based media storage platform.

The platform handles:

- Product Images
- Profile Images
- Avatar Images
- Try-On Images
- Fashion Assets

Upload Lifecycle

Step 1

User uploads image.

Step 2

Multer processes multipart form data.

Step 3

Cloudinary Storage Engine receives file.

Step 4

Image optimization occurs.

Step 5

Cloudinary generates URL.

Step 6

URL stored in MongoDB.

Step 7

Frontend accesses image using CDN URL.

Why Cloudinary?

Scalability

Supports millions of images.

CDN Delivery

Fast worldwide image access.

Automatic Optimization

- Compression
- Resizing
- Format conversion
- WebP generation

Security

- Secure URLs
- Access controls
- Backup management

Image Categories Managed

Product Images

Fashion inventory display.

Profile Images

User identity representation.

Avatar Images

AI-generated avatars.

Try-On Results

Virtual fitting outputs.

20. AI INTEGRATION PLAN

The AI module is the most innovative component of the Raritone platform.

Its objective is to create a personalized fashion experience through machine learning and computer vision.

AI Virtual Try-On System

Goal

Allow users to visualize clothing before purchasing.

Workflow

1. User uploads body image.
2. User selects product.
3. Body landmarks extracted.
4. Garment mapped onto body.
5. AI generates realistic image.
6. Result stored in Cloudinary.
7. URL returned to frontend.

Expected Benefits

- Reduced returns
- Increased customer confidence
- Improved conversion rates

AI Size Recommendation Engine

Input Data

- Height
- Weight
- Chest
- Waist
- Shoulder
- Hip

Output

Recommended Size: M

Confidence: 92%

Benefits

- Better fitting
- Reduced exchanges
- Higher customer satisfaction

AI Fashion Recommendation Engine

Data Sources

- Wishlist
- Cart
- Order History
- Browsing Patterns
- Favorite Categories

Recommendation Examples

Customers who bought this shirt also preferred these jeans.

Algorithms

- Collaborative Filtering
- Content-Based Filtering
- Hybrid Recommendation Models

AI Avatar Generation

Purpose

Create a digital representation of users.

Inputs

- Gender
- Skin Tone
- Body Type

Outputs

- Personalized avatar
- Try-on compatibility model

Future AI Roadmap

Phase 1

Virtual Try-On

Phase 2

Size Recommendation

Phase 3

Fashion Recommendation Engine

Phase 4

AI Stylist Assistant

Phase 5

Voice-Based Fashion Search

Phase 6

Generative Fashion Design Suggestions

21.Team Contributions

- API Development
- Database Design
- JWT Security
- OAuth Integration
- Cloudinary Setup
- Image Management
- Error Handling
- Data Validation
- Security Measures
- Documentation
- Version Control
- API Testing
- Architecture Design
- AI Planning

- Scalability Analysis

22. My contributions

Contributor Information

Name: Nainisha Bandari

Role: Backend Developer

Overview

During the development of the Raritone Backend System, I actively contributed to the design, development, integration, testing, and documentation of several core backend modules. My primary focus was on creating user-centric features that improve personalization and overall user experience within the virtual fashion platform.

I worked extensively on the Profile Management Module and Wardrobe Management Module, which form an essential part of the platform's personalization capabilities. These modules allow users to manage their personal information, preferences, and clothing collections while providing a strong foundation for future AI-driven recommendation systems and virtual styling features. In addition to module development, I participated in API testing, database design validation, GitHub collaboration, and documentation preparation to ensure the successful completion of the backend system.

PROFILE MANAGEMENT MODULE DEVELOPMENT

Objective

The Profile Management Module was developed to provide users with a centralized location where they can manage personal information, preferences, profile images, and account settings.

The goal of this module was to create a personalized environment for every user while ensuring data consistency and secure access to profile information.

Responsibilities

My responsibilities included:

- Designing profile-related database schemas
- Creating profile management APIs
- Implementing profile retrieval functionality
- Developing profile update functionality
- Managing user preference storage
- Integrating profile data with authentication workflows
- Testing profile operations

Implementation Details

I developed APIs that allow authenticated users to retrieve and update their profile information. The module was integrated with JWT authentication middleware to ensure that only authorized users can access and modify their personal data.

The profile system stores information such as:

- User details
- Personal preferences
- Profile image references
- Fashion-related settings

Special attention was given to maintaining data validation and ensuring proper communication between controllers, routes, and database models.

Contribution Impact

This module significantly improves personalization capabilities within the platform by allowing users to customize their

experience. It also serves as a foundation for future recommendation systems that will utilize user preferences to suggest relevant fashion products.

WARDROBE MANAGEMENT MODULE DEVELOPMENT

Objective

The Wardrobe Module was designed to provide users with a digital representation of their personal clothing collection. This feature allows users to organize clothing items and prepares the platform for future outfit recommendation and virtual styling capabilities.

Responsibilities

My contributions included:

- Designing wardrobe database structures
- Developing wardrobe APIs
- Creating wardrobe item management workflows
- Implementing item retrieval operations
- Supporting image storage integration
- Testing wardrobe functionalities

Implementation Details

The module allows users to add, view, update, and delete clothing items from their digital wardrobe.

Each wardrobe item contains important information such as:

- Clothing category
- Item description
- Clothing images
- Fashion attributes
- User ownership references

The APIs were developed using Express.js and MongoDB while maintaining proper relationships between users and wardrobe records.

Data retrieval operations were optimized to ensure efficient performance even when multiple wardrobe items are stored by a user.

Contribution Impact

The wardrobe system enhances user engagement by enabling users to organize and manage their fashion collections digitally. It also establishes a strong foundation for future intelligent styling recommendations and virtual fashion experiences.

USER PROFILE MANAGEMENT

Objective

The User Profile Management Module was developed to provide a centralized system where users can manage their personal information, profile details, and account preferences. The primary objective of this module is to create a personalized user experience by allowing individuals to maintain accurate and up-to-date information within the platform.

This module acts as a bridge between user identity and platform personalization, ensuring that user-specific data can be securely stored, accessed, and modified whenever required. It also establishes the foundation for future features such as personalized recommendations, customized shopping experiences, and intelligent user analytics.

Responsibilities

My responsibilities during the development of the User Profile Management Module included:

- Designing the profile database schema
- Developing profile creation functionality
- Implementing profile retrieval APIs
- Creating profile update operations
- Managing user preference storage
- Integrating profile data with user authentication
- Validating profile-related data
- Testing profile management workflows
- Ensuring secure access to profile information
- Supporting future personalization features

Implementation Details

The User Profile Management Module was designed to maintain comprehensive information about each user within the platform. The module enables authenticated users to view and update their profile information through secure RESTful APIs.

The profile system stores important user-related information such as personal details, profile images, and individual preferences. The module was integrated with the authentication system to ensure that users can only access and modify their own profile information.

During implementation, proper validation mechanisms were introduced to verify incoming data before database operations were performed. This helped maintain data accuracy and prevented invalid information from being stored.

The module was developed using Express.js controllers, MongoDB collections, and Mongoose schemas. API endpoints were created to support profile retrieval, profile updates, and preference management operations. Error handling mechanisms were also implemented to ensure smooth communication between the client application and backend services.

To improve maintainability, the module follows a modular architecture where routes, controllers, models, and validation logic are separated into independent components. This structure makes future enhancements easier to implement and reduces development complexity.

Contribution Impact

The User Profile Management Module plays a critical role in enhancing user experience throughout the platform. By allowing users to maintain their personal information and preferences, the system creates a more personalized and user-friendly environment.

The module improves user engagement by ensuring that account information remains accurate and accessible. It also supports future platform enhancements by providing essential user data that can be utilized by recommendation engines, virtual styling systems, avatar generation modules, and AI-driven personalization features.

Furthermore, the profile management functionality contributes to the overall scalability of the platform by establishing a structured and reliable framework for handling user-specific information. Its implementation strengthens the platform's ability to deliver

customized experiences while maintaining data security, consistency, and ease of management.

USER PREFERENCE MANAGEMENT

Objective

To improve personalization across the platform, I contributed to implementing user preference management functionality.

Responsibilities

- Designing preference-related structures
- Managing preference storage
- Integrating preferences with user profiles
- Supporting future recommendation workflows

Implementation Details

The preference management system stores information regarding user interests and fashion choices. These preferences can later be utilized by recommendation engines to provide customized product suggestions.

The architecture was designed to support future AI-based recommendation systems without requiring major modifications to the backend structure.

Contribution Impact

This functionality improves user experience by enabling personalized interactions and supporting intelligent shopping experiences in future versions of the platform.

API DEVELOPMENT

Throughout the project, I actively participated in developing and testing RESTful APIs.

Responsibilities

- Route development
- Controller implementation
- API response management
- Error handling integration
- Endpoint testing

Development Approach

While implementing APIs, I followed RESTful architecture principles to ensure consistency and maintainability.

Every endpoint was designed to:

- Accept validated input
- Process business logic efficiently
- Interact with MongoDB collections
- Return structured JSON responses
- Handle exceptions gracefully

This approach improved overall reliability and developer experience.

Documentation Contributions

Documentation was an important part of the project because it helps future developers understand the architecture and functionality of the system.

Responsibilities

- Module documentation
- API documentation
- Workflow explanations
- Testing documentation
- Contribution reporting

Contribution

I participated in preparing technical documentation that explains the functionality, architecture, workflows, and implementation details of the modules I developed.

This documentation improves maintainability and supports future project enhancements.

23.GitHub Collaboration and Team Coordination

During the project development process, I actively collaborated with team members using GitHub.

Responsibilities

- Source code management
- Branch coordination
- Code integration
- Pull request reviews
- Version control management

Contribution

I regularly synchronized module updates, resolved integration conflicts, and ensured compatibility between different backend components.

This collaboration helped maintain project consistency and facilitated smooth team development.

Skills Applied During Development

Throughout the project, I applied several backend development skills including:

- Node.js Development
- Express.js Development
- MongoDB Database Management
- Mongoose ODM

- REST API Development
- JWT Authentication
- Middleware Integration
- Postman API Testing
- GitHub Version Control
- Backend Architecture Design
- Database Relationship Management
- Technical Documentation

Key Achievements

Some of my major achievements during the project include:

- Successfully developed the Profile Management Module.
- Successfully developed the Wardrobe Management Module.
- Implemented user preference management functionality.
- Developed and tested multiple RESTful APIs.
- Contributed to database relationship validation.
- Participated in integration and testing activities.
- Assisted in project documentation preparation.
- Supported future AI-readiness through scalable design decisions.
- Collaborated effectively with team members using GitHub.

24.Summary

As a Backend Developer for the Raritone Backend Project, I contributed significantly to the development of personalization-focused modules including Profile Management and Wardrobe Management. My work involved designing database structures, implementing RESTful APIs, validating data operations, testing functionality, documenting system behavior, and collaborating with team members throughout the development lifecycle.

These contributions helped establish a scalable, maintainable, and user-focused backend architecture that supports current platform requirements while providing a strong foundation for future enhancements such as AI-powered recommendations, intelligent wardrobe analysis, virtual styling assistance, and advanced personalization capabilities.

25.Conclusion

The Raritone Backend Project successfully provides a secure, scalable, and well-structured foundation for a virtual fashion platform. The system integrates essential functionalities such as user authentication, profile management, product management, wardrobe management, shopping cart operations, order processing, avatar management, and virtual try-on support within a single backend architecture.

The project was developed using modern technologies including Node.js, Express.js, MongoDB, and Cloudinary, ensuring efficient performance, secure data handling, and maintainable code structure. Security features such as JWT authentication, role-based access control, input validation, and error handling mechanisms were implemented to protect user data and ensure reliable system operation.

A key strength of the project is its focus on personalization and future scalability. Features like user profiles, body measurements, wardrobes, and avatars provide a strong foundation for future AI-powered recommendations, virtual styling, and intelligent fashion experiences. The modular architecture also allows new features and technologies to be integrated with minimal changes to the existing system.

Overall, the project demonstrates practical knowledge of backend development, API design, database management, security implementation, cloud integration, and software engineering principles.